

D0 High-Level Design

Verified Student Sublease Marketplace

Course	CS5001 Senior Design I
Assignment	Assignment 5: High-Level System Design (Design D0)
Team members	Danvanth Vanganuri, Jovan Jose Asker Fredy, Mohamed Aidja
Repository location	Design_Diagrams/D0_High_Level_Design.pdf (this document) Design_Diagrams/D0_Block_Diagram.drawio Design_Diagrams/D0_DataFlow_1_Verify_Account.drawio, D0_DataFlow_2_Publish_Hold_Review.drawio, D0_DataFlow_3_Search_And_Message.drawio, D0_DataFlow_4_Approve_Subleaser.drawio
Inputs to this design	Assignment 4 user stories US-01 to US-05, use cases UC-01 and UC-02, acceptance criteria AC-01.1 to AC-02.2

1. Title, Goal Statement, and Conventions

Project title: Verified Student Sublease Marketplace

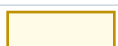
Goal statement: Let UC students fill or find term-length subleases safely by limiting the marketplace to verified @mail.uc.edu students, matching listings to exact co-op dates, holding discriminatory listings for review, and giving property managers an approve/reject step before move-in.

Basic input: listing details and sign-in requests from subleasing students, co-op start and end dates and messages from sublessee students, approve or reject decisions from property managers, and release or reject decisions from the Fair Housing reviewer.

Basic output: Fair-Housing-screened listings visible only to verified UC students whose co-op dates they cover, email notifications to students and property managers, and a recorded approval state for every proposed sublesser.

Diagram conventions

Every diagram in this document (the block diagram and all four data-flow diagrams) carries the project title, the goal statement, and its own legend drawn on the diagram.

Symbol	Meaning
	Blue rounded rectangle, label Cn: a component the team designs and builds.
	Gray dashed rectangle, label En: an external system we depend on but do not build (email provider, recipient mailboxes, managed database).
	Yellow ellipse: an actor, a person who uses the system.
	Dotted outline (block diagram only): deployment boundary. C2 to C8 run inside one deployable backend.
	Green open-sided rectangle (data-flow diagrams only): a data store, rows at rest in E3.
	Arrow labelled Ix (block diagram): interface Ix from Part 4. It points from the side that initiates to the side that receives; replies return on the same interface.
	Double-headed arrow: both sides initiate (for example, the E1 status webhook) or data moves both ways. An arrow from the dotted boundary means every component inside that Part 4 lists for the interface uses it (I15).
	Arrow labelled with data (data-flow diagrams): the data crossing it, its form at that point (raw input, JSON, queue row, stored row, event, email), and the interface ID it travels on.
	Note box (data-flow diagrams): the timing budgets and rules that the requirements set for that flow.

2. Block Diagram (D0)

Verified Student Sublease Marketplace | Design D0: Block Diagram

Goal: Let UC students fill or find term-length subleases safely by limiting the marketplace to verified @mail.uc.edu students, matching listings to exact co-op dates, holding discriminatory listings for review, and giving property managers an approve/reject step before move-in.

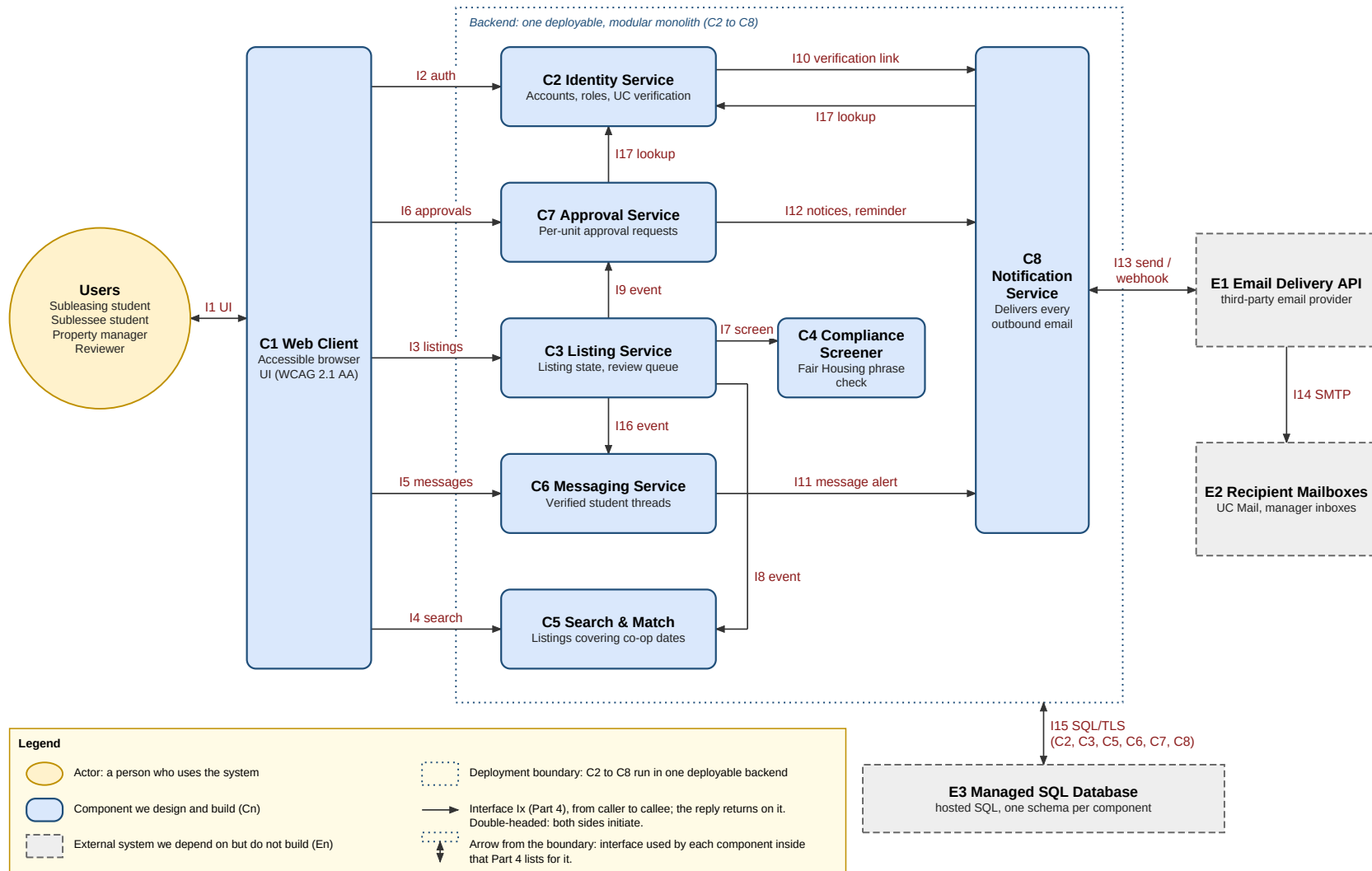


Figure 1. D0 block diagram: eight built components (C1 to C8), three external systems (E1 to E3), seventeen interfaces (I1 to I17).

The browser client (C1) is the only entry point for users. Each backend component owns one capability and is reached through a named interface. Side effects of publishing a listing (index entry, owner copy for messaging, manager notice) travel as events (I8, I16, I9), so a slow email provider can never block a listing submission. Account and role data lives only in C2; other components ask for it through I17 or read it from the signed session token.

3. Component Responsibility Table

“In” lists interfaces on which the component receives a request or event; “Out” lists interfaces it initiates. Owners named here become the task owners in Assignment 8.

Component	Responsibility (one sentence)	In	Out	Stories	Primary owner
C1 Web Client	Presents every screen as a browser UI that works by keyboard or screen reader.	I1	I2, I3, I4, I5, I6	US-02, US-04 (UI for all stories)	Mohamed Aidja
C2 Identity Service	Owns every account record: identity, role, verification status, session token.	I2, I17	I10, I15	US-01, US-03 (manager login), US-05 (reviewer role)	Danvanth Vanganuri
C3 Listing Service	Owns each listing's lifecycle state: published, held for review, or rejected.	I3	I7, I8, I9, I15, I16	US-01, US-05	Jovan Jose Asker Fredy
C4 Compliance Screener	Flags Fair Housing preference language in a listing description.	I7	none (stateless)	US-05	Jovan Jose Asker Fredy
C5 Search & Match	Returns published listings whose availability covers a requested date range.	I4, I8	I15	US-02, US-04	Jovan Jose Asker Fredy
C6 Messaging Service	Delivers messages between verified students about a published listing.	I5, I16	I11, I15	US-01, US-04	Mohamed Aidja
C7 Approval Service	Owns each unit's approval record: registered manager, proposed sublease, decisions.	I6, I9	I12, I15, I17	US-03	Danvanth Vanganuri
C8 Notification Service	Delivers every outbound email requested by another component.	I10, I11, I12, I13 (status webhook)	I13 (send), I15, I17	US-01, US-03	Danvanth Vanganuri

External systems (not built)

System	Role	In	Out	Integration owner
E1 Email Delivery API	Third-party transactional email provider; sends our emails, reports delivery status by webhook.	I13 (send)	I13 (status webhook), I14	Danvanth Vanganuri
E2 Recipient Mailboxes	UC Mail (@mail.uc.edu, Microsoft 365) and manager or reviewer inboxes. Must receive well-formed, rate-limited email.	I14	none	n/a (we only send)
E3 Managed SQL Database	Hosted relational database; each component owns its own schema and no component reads another's tables.	I15	none	Jovan Jose Asker Fredy (migrations)

Story-to-component traceability

Story	Components that make it happen	Where it shows in D0
US-01	C2 (verification link, 12-month status), C3 (listing), C6 (verified-only contact), C8 (link and alert emails)	I2, I3, I5, I10, I11; Flows 1, 2, 3
US-02	C5 (date-coverage match), C1 (search screen)	I4, I8; Flow 3
US-03	C7 (approval state, reminder, registered manager), C8 (notices), C2 (manager login)	I2, I6, I9, I12, I17; Flows 2, 4
US-04	C1 (accessible UI), C5 (search), C6 (message)	I1, I4, I5; Flow 3
US-05	C4 (screening), C3 (hold and review queue), C2 (reviewer role)	I3, I7; Flow 2

Every component traces to at least one story and every story to at least one component. Screening is split out of C3 into C4, indexing into C5, and account data into C2, so no component's responsibility needs the word “and”. Account creation and login are covered by US-01 precondition 1 through the one-time link on I2 (decision D-09).

4. Interface Specification Table

Types: string, int, bool, date (ISO 8601 yyyy-mm-dd), datetime (ISO 8601 UTC), UUID, enum{...}. "?" marks an optional field. Every client request (I2 to I6) except the first I2 call carries the session token issued on I2 in an Authorization header. The handler of each error is named in parentheses at the end of the cell.

ID	From → To	Inputs (name: type)	Outputs (name: type)	Format	Protocol	On error (handled by)
I1	Users → C1	address: string; rentCents: int; startDate, endDate: date; description: string ≤ 2000; search dates; message body; proposal or decision + reason; review decision	Rendered pages, status messages, flagged phrase, result lists	HTML with ARIA labels and landmarks	Browser over HTTPS; WCAG 2.1 AA, fully keyboard operable	Field errors are announced through an ARIA live region and focus moves to the first invalid field. (C1)
I2	C1 → C2	email: string; token?: string (from the one-time link)	status: enum{UNVERIFIED, PENDING, VERIFIED}; verifiedAt?: datetime; sessionToken?: string (signed; claims userId, role, verified, verifiedAt, exp)	JSON	HTTPS REST: POST /auth/link-requests, POST /auth/link-confirmations. One link is both sign-up and login.	Domain not @mail.uc.edu and no seeded manager or reviewer account → 422 NOT_ELIGIBLE. Token older than 24 h → 410 EXPIRED, account stays unverified, C1 offers a new link (AC-01.2). Unknown or used token → 400 INVALID_TOKEN. More than 3 requests/hour → 429 + Retry-After, protecting UC Mail. (C2)
I3	C1 → C3	Create: address: string; rentCents: int; startDate: date; endDate: date; description: string ≤ 2000. Reviewer: listingId: UUID; decision: enum{RELEASE, REJECT}; note?: string	listingId: UUID; status: enum{PUBLISHED, HELD_FOR_REVIEW, REJECTED}; flaggedPhrases: string[]. Review queue: [{listingId, description, flaggedPhrases}]	JSON	HTTPS REST: POST /listings, GET /listings, POST /review-queue, POST /listings/{id}/review (reviewer role only)	Token missing, unverified, or verifiedAt older than 12 months → 403 VERIFICATION_REQUIRED, C1 starts I2. endDate ≤ startDate or rentCents ≤ 0 → 422, no record created (AC-01.4). Active listing at the same address → 409. Caller lacks reviewer role → 403 FORBIDDEN. Review of a listing that is not HELD_FOR_REVIEW → 409. (C3)
I4	C1 → C5	startDate: date; endDate: date; maxRentCents?: int; page: int	results: [{listingId: UUID, addressSummary: string, rentCents: int, availableStart: date, availableEnd: date}]; total: int	JSON	HTTPS REST: GET /listings/search	Unverified caller → 403 (listings are visible to verified students only). endDate ≤ startDate → 400. No match → 200 with an empty list, not an error. (C5)
I5	C1 → C6	listingId: UUID; body: string (1 to 2000 chars)	threadId: UUID; messageId: UUID; sentAt: datetime	JSON	HTTPS REST: POST /listings/{id}/messages, GET /threads	Sender unverified, or listing not PUBLISHED in C6's copy (from I16) → 403. Empty or oversize body → 422. (C6)
I6	C1 → C7	Proposal: listingId: UUID, sublesseeUserId: UUID. Decision: requestId: UUID, decision: enum{APPROVE, REJECT}, reason?: string (required for REJECT)	requestId: UUID; status: enum{PENDING, APPROVED, REJECTED}; decidedAt: datetime	JSON	HTTPS REST: POST /approval-requests, POST /approval-requests/{id}/decision	Sublessee not verified (checked through I17) → 422. Caller is not the listing owner (proposal) or the unit's registered manager (decision) → 403. Decision on a non-PENDING request → 409, keeping exactly one state. REJECT without reason → 422. (C7)
I7	C3 → C4	text: string	flagged: bool; matches: [{phrase: string, category: enum{RACE, RELIGION, FAMILIAL_STATUS, ...}, offset: int}]; listVersion: string	Typed in-memory object	In-process function call	Exception or no answer within 2 s → C3 fails closed: status HELD_FOR_REVIEW, reason SCREENER_UNAVAILABLE. A listing is never published unscreened. (C3)
I8	C3 → C5	event: listing.published; eventId: UUID; listingId: UUID; rentCents: int; availableStart, availableEnd: date; addressSummary: string	ack: bool	JSON event row	Transactional outbox: written in the same DB transaction as the status change; relay polls every 5 s	Handler failure → event stays in the outbox and is retried after 5 s, 30 s, 2 min, then logged as FAILED for the owner. C5 upserts by eventId, so retries are idempotent. (C3 relay)
I9	C3 → C7	event: listing.published; eventId: UUID; listingId: UUID; ownerId: UUID; address: string; availableStart, availableEnd: date	ack: bool	JSON event row	Transactional outbox (as I8)	Retry as I8. No manager registered for the address → C7 records UNREGISTERED_UNIT and skips the notice; the listing stays published. (C7)
I10	C2 → C8	templateId: LOGIN_LINK; to: string (email); params: {link: URL, expiresAt: datetime}; idempotencyKey: string	notificationId: UUID; state: QUEUED	Typed object	In-process call that inserts a notification-queue row	Enqueue fails → C2 returns 503 to C1 and does not mark the account PENDING. (C2)
I11	C6 → C8	templateId: NEW_MESSAGE; toUserId: UUID; params: {listingId, senderFirstName}; idempotencyKey	notificationId: UUID	Typed object	In-process enqueue	Enqueue fails → the message is still stored and visible in the thread; the alert failure is logged. (C6)
I12	C7 → C8	templateId: enum{NEW_LISTING, SUBLEASE_PROPOSED, DECISION, REMINDER}; toUserIds: UUID[]; params: object; idempotencyKey	notificationId: UUID	Typed object	In-process enqueue inside C7's DB transaction	Enqueue fails → C7 rolls back the state change and returns 503; nothing is recorded without its notice. The reminder job sets reminderSent only after enqueue, so at most one reminder is sent (AC-02.2). (C7)

ID	From → To	Inputs (name: type)	Outputs (name: type)	Format	Protocol	On error (handled by)
I13	C8 ↔ E1	Send: to, from, subject, html, text: string; idempotencyKey. Status webhook in: messageId: string, event: enum{DELIVERED, DEFERRED, BOUNCED}, timestamp	Send result: messageId: string, accepted: bool	JSON	HTTPS REST with API key; signed HTTPS webhook back	5xx or timeout → retry at 30 s, 60 s, 120 s (fits the 5-minute budget), then FAILED and logged. 4xx → FAILED, no retry. BOUNCED → recipient marked undeliverable. Bad webhook signature → 401, discarded. (C8)
I14	E1 → E2	Email message	Delivery status (reported to C8 through the I13 webhook)	RFC 5322 MIME	SMTP over TLS	Deferral or bounce is retried by E1; the final status reaches C8 over I13. (E1, reported to C8)
I15	C2, C3, C5, C6, C7, C8 ↔ E3	Parameterized SQL statements	Rows; affected-row count	Relational rows	Database wire protocol over TLS, connection pool	Connection loss or timeout → transaction rolled back so no partial listing or request exists; the caller returns 503 with a retry hint; the pool reconnects. (calling component's data-access layer)
I16	C3 → C6	event: listing.published; eventId: UUID; listingId: UUID; ownerId: UUID; addressSummary: string	ack: bool	JSON event row	Transactional outbox (as I8)	Retry as I8. C6 upserts its copy of (listingId, ownerId) by eventId. Until the copy exists, messages to that listing get 403 and the sender can retry. (C3 relay)
I17	C7, C8 → C2	userIds: UUID[]	users: [{userId: UUID, email: string, role: enum{STUDENT, MANAGER, REVIEWER}, verified: bool, firstName: string}]	Typed object	In-process read-only lookup (C8 uses email; C7 uses verified and role only)	Unknown userId → omitted from the result; the caller treats it as undeliverable (C8) or ineligible (C7). Database failure → C8 retries 3 times then marks FAILED; C7 returns 503. (calling component)

Example payload (I3, create a listing)

POST /listings Authorization: Bearer <sessionToken from I2>

```
{
  "address": "100 Sample St, Apt 3B, Cincinnati, OH 45219",
  "rentCents": 85000,
  "startDate": "2027-01-06",
  "endDate": "2027-04-30",
  "description": "Furnished room, 10-minute walk to campus, utilities included."
}
```

201 Created

```
{ "listingId": "6f1c2e7a-4b8d-4c1e-9a55-0d3e2f7b9c10", "status": "PUBLISHED", "flaggedPhrases": [] }
```

Same interface, held case (AC-01.3)

POST /listings (description contains a phrase on the Fair Housing flag list)

201 Created

```
{ "listingId": "0b7d9a31-5c2e-4f08-8e6a-71c4d2a9e5b3", "status": "HELD_FOR_REVIEW", "flaggedPhrases": ["<flagged phrase>"] }
```

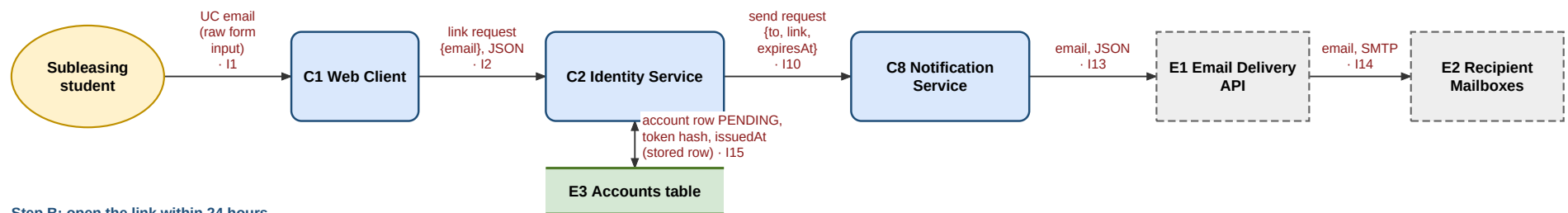
5. Data-Flow Diagram

The system has four distinct flows, so all four are drawn. Flow 1 verifies an account (UC-01 steps 2 to 6). Flow 2 follows a listing from raw form input to a validated record, a stored row, and events that reach the index, the messaging copy, and the property manager's inbox, including the hold-and-review branch for Fair Housing flags (UC-01 steps 7 to 10, US-05). Flow 3 follows a sublessee's co-op dates to a result list, and a message to the listing owner (US-02, US-04). Flow 4 follows a proposed sublesser to a pending row, a manager decision, and notices to both students (UC-02). Each arrow carries the data and its form at that point; the note box on each diagram lists the timing budget that a requirement sets.

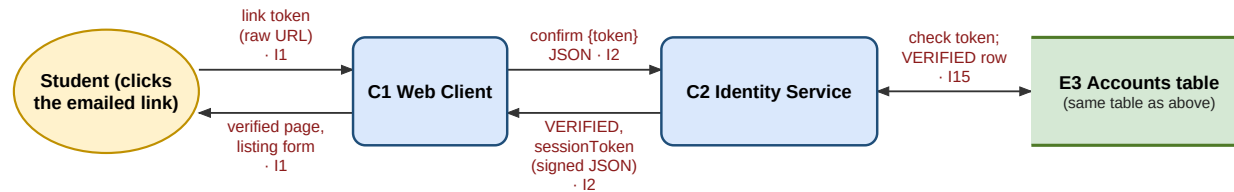
Verified Student Sublease Marketplace | Data Flow 1 (UC-01 steps 2 to 6, US-01): verify a student account

Goal: Let UC students fill or find term-length subleases safely by limiting the marketplace to verified @mail.uc.edu students, matching listings to exact co-op dates, holding discriminatory listings for review, and giving property managers an approve/reject step before move-in.

Step A: request the one-time link



Step B: open the link within 24 hours



Timing budgets and rules

UC-01 step 4: the link is valid for 24 h. C2 stores issuedAt and checks it on I2. An expired link returns 410, the account stays UNVERIFIED, and no listing is published (AC-01.2).

At most 3 link requests per hour per account, so E2 is not flooded. VERIFIED lasts 12 months (UC-01 A1). The session token carries role and verified status and expires after 24 h (D-03).

Legend

Actor: a person who uses the system

Component we design and build (Cn)

External system we depend on but do not build (En)

Data store: rows at rest in E3 (each component has its own schema)

Data flow. The label gives the data crossing it, its form at that point, and the interface Ix it travels on (Part 4).

Note box: timing budgets and rules from the requirements

Figure 2. Data Flow 1: verify a student account with a one-time link.

Verified Student Sublease Marketplace | Data Flow 2 (UC-01 steps 7 to 10, US-01, US-05): publish, hold, or review a listing

Goal: Let UC students fill or find term-length subleases safely by limiting the marketplace to verified @mail.uc.edu students, matching listings to exact co-op dates, holding discriminatory listings for review, and giving property managers an approve/reject step before move-in.

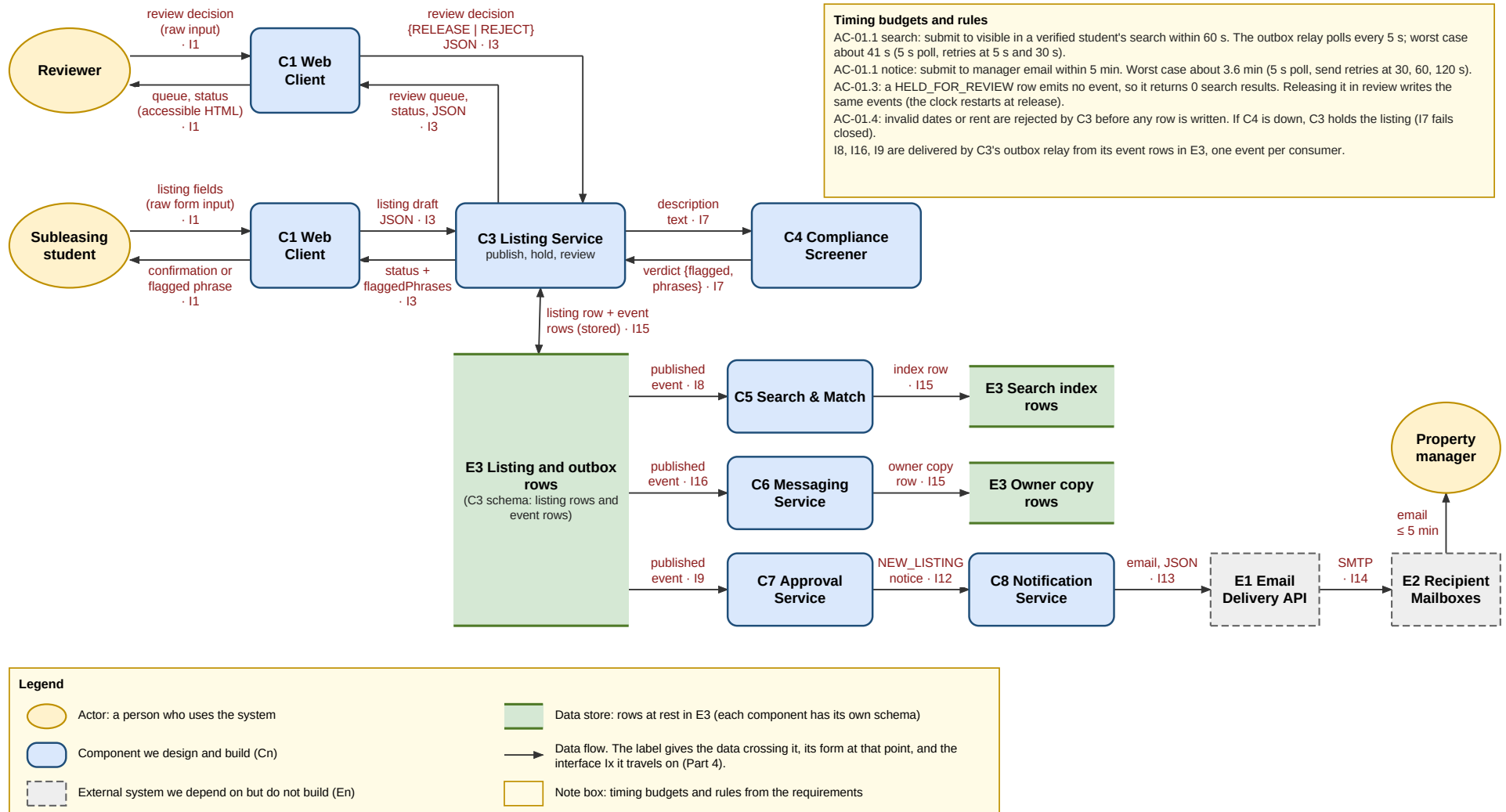
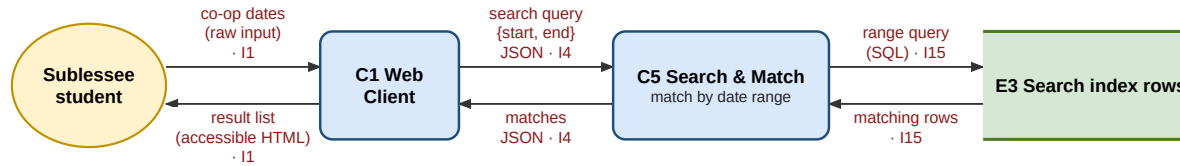


Figure 3. Data Flow 2: publish a listing, hold it for Fair Housing review, release it, and fan out its events.

Verified Student Sublease Marketplace | Data Flow 3 (US-02, US-04, US-01 contact): find a listing by co-op dates and message the owner

Goal: Let UC students fill or find term-length subleases safely by limiting the marketplace to verified @mail.uc.edu students, matching listings to exact co-op dates, holding discriminatory listings for review, and giving property managers an approve/reject step before move-in.

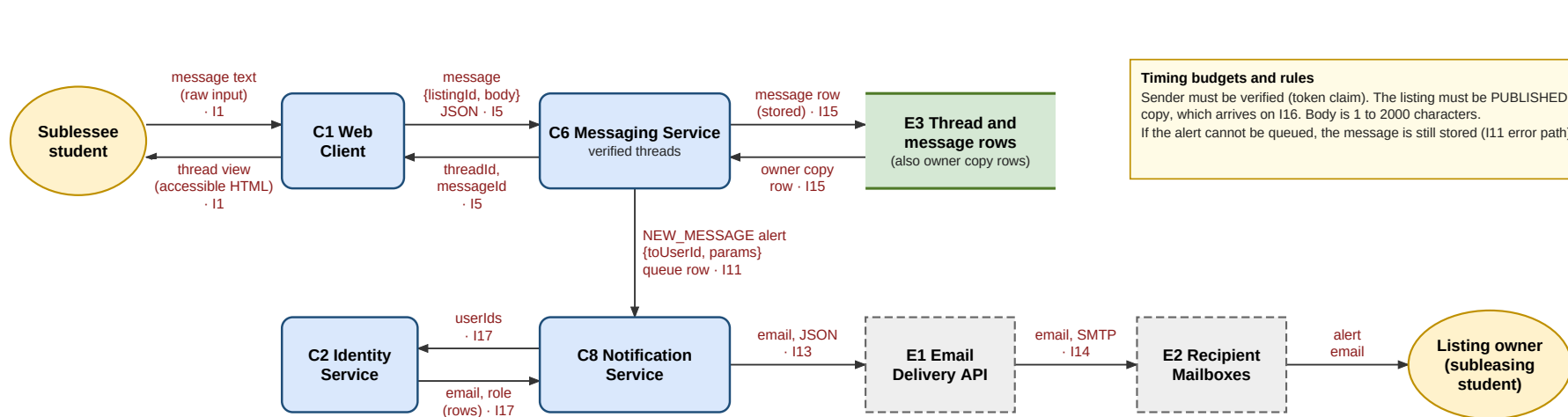
Step A: search by co-op dates



Timing budgets and rules

D-06: a listing matches when availableStart ≤ co-op start and availableEnd ≥ co-op end. Only PUBLISHED listings are indexed, so held listings return 0 results (AC-01.3). An unverified caller gets 403.

Step B: message the listing owner



Timing budgets and rules

Sender must be verified (token claim). The listing must be PUBLISHED in C6's owner copy, which arrives on I16. Body is 1 to 2000 characters. If the alert cannot be queued, the message is still stored (I11 error path).

Legend

- Actor: a person who uses the system
- Component we design and build (Cn)
- External system we depend on but do not build (En)
- Data store: rows at rest in E3 (each component has its own schema)
- Data flow. The label gives the data crossing it, its form at that point, and the interface Ix it travels on (Part 4).
- Note box: timing budgets and rules from the requirements

Figure 4. Data Flow 3: find a listing by co-op dates and message the listing owner.

Verified Student Sublease Marketplace | Data Flow 4 (UC-02, US-03): approve or reject a proposed sublesser

Goal: Let UC students fill or find term-length subleases safely by limiting the marketplace to verified @mail.uc.edu students, matching listings to exact co-op dates, holding discriminatory listings for review, and giving property managers an approve/reject step before move-in.

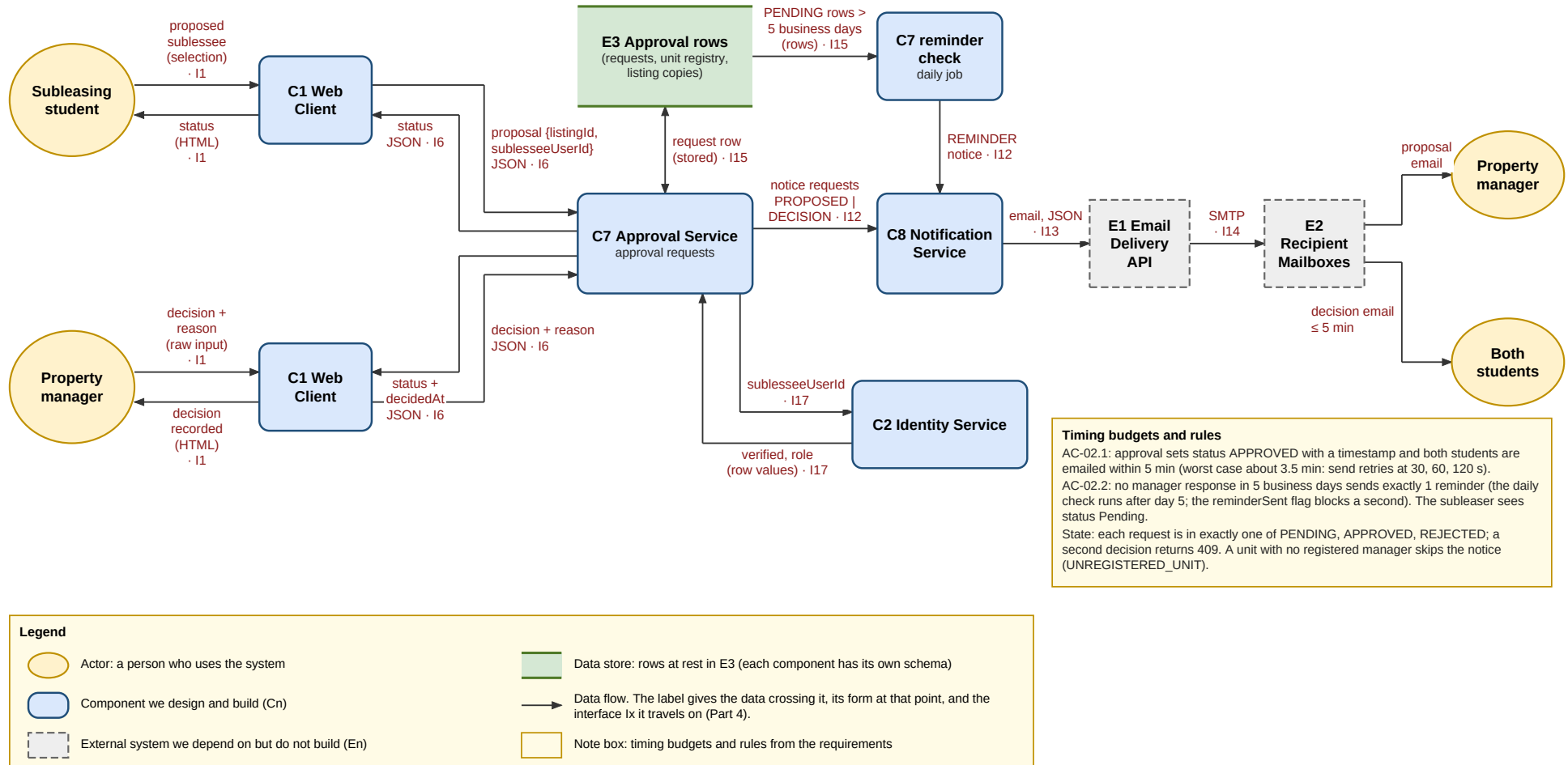


Figure 5. Data Flow 4: property manager approves or rejects a proposed sublesser.

Timing budgets set by requirements

Requirement	Budget	Stages on the path	Worst case in this design
AC-01.1 (search)	Submit → visible in search ≤ 60 s	C3 commit (row + outbox event) → outbox relay poll → I8 → C5 index upsert	5 s poll + retries at 5 s and 30 s ≈ 41 s. A third retry (2 min) would miss the budget; that is a failure case, logged as FAILED.
AC-01.1 (notice)	Submit → manager email ≤ 5 min	Outbox relay → I9 → C7 → I12 → C8 → I17 → I13 → E1 → I14	5 s poll + send retries at 30, 60, 120 s ≈ 3.6 min
AC-02.1	Approval → both students emailed ≤ 5 min	C7 commit → I12 → C8 → I17 → I13 → E1 → I14	Send retries at 30, 60, 120 s ≈ 3.5 min
AC-02.2	No manager response in 5 business days → exactly 1 reminder	C7 daily reminder check → I12 REMINDER → C8	Sent on the first daily run after day 5; the reminderSent flag prevents a second
UC-01 step 4 (AC-01.2)	Verification link valid 24 h	C2 stores issuedAt; checked when the link is opened on I2	Expired link → 410, account stays unverified
AC-01.3	Held listing returns 0 results in any other user's search	C3 writes a HELD_FOR_REVIEW row and no event; C5 only indexes events	A held listing is never indexed until a reviewer releases it

6. Architecture Pattern and Justification

We use a client-server pattern at the top level, a layered pattern inside the backend, and a pipeline for the listing publication path. Each pattern governs a different part of the system:

Pattern	Governs	How it appears in D0
Client-server	C1 ↔ backend (I1 to I6)	One browser client calls the backend over HTTPS. The backend holds all state and all rules, so a client cannot bypass verification, screening, or the manager's approval.
Layered	Inside the backend (C2 to C8) and E3	Request-handling layer (HTTP endpoints) → service layer (C2 to C8 modules) → data-access layer → E3. Calls only go downward; a module reaches another only through its named interface (I7 to I12, I16, I17).
Pipeline	Listing publication: C3 → C4 → E3 → events to C5, C6, C7, then C7 → C8 → E1 as the notice branch	Validate → screen → persist (or hold for review) → index and notify. Each stage has one input and one output form (Data Flow 2), and a failed stage stops the item without publishing it.

Justification against the five criteria

Criterion	Why the chosen combination fits
Fit to the problem	The product is a web marketplace: forms, search, messaging, and a small approval workflow around one relational data set. Client-server plus layers is the standard shape for that. Publication is a strict sequence of checks, which is what a pipeline expresses, and it lets the Fair Housing screen sit in its own stage (C4) that can be replaced later without touching C3.
Team skills	The three team members bring full-stack web, SQL database, and API-integration experience from co-ops and earlier projects, so a browser client, a REST backend, and a relational database are familiar ground. One repository and one deployable keep three people in sync, while module boundaries (C2 to C8) let each owner work independently against the Part 4 interfaces. Distributed services would add skills the team has not practiced (service discovery, distributed tracing).
Performance and timing	The binding budgets are generous (60 s to search, 5 min to email, Part 5). In-process calls between modules avoid network hops. The outbox decouples slow email delivery from listing submission, and its worst cases (about 41 s and 3.6 min) still meet both budgets.
Scalability	Demand is bounded by UC students subleasing for a co-op term: low thousands of users and a few hundred active listings per term. One backend instance plus a managed database covers this. If load grows, the backend can be replicated behind a load balancer, and a module with a clean interface (C8 first) can be extracted.
Hardware constraints	None beyond commodity cloud hosting. Users bring their own devices; the only device-level need is assistive technology (screen readers), handled in C1's presentation layer, not by hardware.

Constraints reflected in the pattern (Week 3 constraint areas)

Constraint area	Effect on the pattern
Economic	Budget: one deployable and one managed database fit a free or student hosting tier, and email goes through a provider with a free tier (D-07). Eight separately hosted services would not.
Security	Privacy: listings, messages, and approval records sit behind server-side verification checks. UC email addresses are never shown to other users, only C8 reads them (I17), and each component reads only its own schema. Sessions use short-lived signed tokens (D-03).
Legal	Fair Housing Act: a screening stage (C4) and a human review queue sit in front of publication. Fail-closed behavior on I7 means a listing is never published unscreened.
Social and Ethical	Accessibility: the layered split isolates presentation in C1, where WCAG 2.1 AA is enforced for the US-04 search-and-message path. Verified-only contact protects students from strangers.
Hosting	A single cloud host and a managed SQL database; no servers the team must patch.

Rejected patterns

- **Microservices:** rejected. Eight independently deployed services would multiply hosting cost, add network failure modes to every interface, and require service discovery, distributed logging, and per-service deployment pipelines that a three-person, one-semester team cannot maintain. The modular monolith keeps the same component boundaries without those costs (D-01).
- **Embedded sensor-actuator:** rejected. The system has no sensors, actuators, or device hardware; its inputs are human form submissions.
- **Pure pipeline for the whole system:** rejected. Search, messaging, and approval are interactive request-response exchanges, not staged transformations; forcing them into a pipeline would add latency and complexity with no benefit.

7. Decision Log

Each entry records the decision, the alternatives considered, and why the chosen option won. These entries are reused as design rationale in D2 and the Week 9 technical specification.

ID	Decision	Alternatives considered	Why the chosen option won
D-01	Deploy the backend as one modular monolith (C2 to C8 as modules).	Microservices; serverless functions per endpoint.	Fits the hosting budget and team size, removes network failures between components, and keeps interface boundaries so a module can be extracted later. Serverless cold starts and per-function configuration add work without a scaling need.
D-02	Verify students with a one-time link to their @mail.uc.edu address, valid 24 h.	UC single sign-on (SAML/Shibboleth); trust the email domain at sign-up; upload of a student ID card.	Proves mailbox ownership and matches UC-01 exactly. SSO needs UC IT registration that cannot be guaranteed within the semester. A domain check alone is spoofable. ID uploads create a privacy risk and need manual review.
D-03	Carry verification status and role as claims in a signed 24 h session token that each module checks locally. The only runtime lookup of another component's account data is I17 (contact email, sublessee status).	Each module calls C2 on every request.	Removes a C2 call from every search and message, so C2 is not a single point of failure for them. Trade-off: a revoked verification lasts until the token expires; verifiedAt enforces the 12-month re-verification rule (UC-01 A1).
D-04	Screen listings with a versioned Fair Housing phrase list; flagged listings go to a human review queue.	Machine-learning text classifier; manual review of every listing.	Deterministic and testable with a fixed phrase list (US-05 INVEST note), and it can show the student the exact phrase (AC-01.3). A classifier needs labelled data and cannot explain misses. Reviewing every listing would break the 60-second publication budget. A classifier can later sit behind the same I7 interface.
D-05	Deliver side effects of publishing (index, manager notice, message copy) through a transactional outbox.	Synchronous calls inside the submit request; a hosted message broker.	A submission never fails because the email provider is slow, and no event is lost because the event row is written in the same transaction as the status change. A 5-second poll meets both timing budgets. A broker is another hosted system to pay for and operate.
D-06	Match listings when availability fully covers the co-op term (availableStart ≤ start and availableEnd ≥ end).	Any date overlap; overlap with a tolerance of ±N days.	US-02 asks for subleases that cover the exact co-op dates; overlap would return listings that leave the student without housing for part of the term. A tolerance option can be revisited in D1 if test users ask for it.
D-07	Send email through a third-party transactional email API (E1).	Run our own SMTP server; relay through UC's mail system.	Better deliverability into UC's Microsoft 365 spam filtering, delivery and bounce webhooks for error handling, and a free tier. Self-hosted SMTP is often blocked or junk-filtered; a UC relay needs IT approval.
D-08	Build one responsive web client (C1).	Native iOS and Android apps.	One codebase for three developers, no app-store review, and screen-reader support through standard semantic HTML and ARIA, which is what US-04 needs.
D-09	Use passwordless one-time links for sign-up and login. Property-manager and reviewer accounts are seeded by the team for now.	Username and password; self-service manager registration with document checks.	Reuses the verification mechanism already in D-02, so no password storage, reset flow, or credential-leak risk to build. UC-02 and US-05 need a manager and a reviewer to exist, not a sign-up flow for them; self-service manager registration is deferred to D1.
D-10	Keep the Fair Housing review queue inside C3, gated by a REVIEWER role claim.	A separate review service; reviewers edit the database directly.	C3 already owns the HELD_FOR_REVIEW state, so releasing or rejecting a listing is one state change with its events in one transaction. A separate service would need a new interface and a copy of the listing. Direct database edits would bypass the outbox and skip the index and notice.
D-11	C7 owns the unit registry (address → registered manager), seeded by the team, and keeps a copy of each published listing's owner and address from I9.	Store the registry in C2 or C3; query C3 on every approval request.	Keeps the 'no component reads another's tables' rule: C7 checks ownership and the registered manager from its own schema. A registry in C2 or C3 would add a lookup interface for a read-mostly table.
D-12	One outbox event (listing.published) fans out to three consumers: C5 (index), C6 (owner copy), C7 (manager notice).	C6 and C7 query C3 when they need listing data; C3 calls each consumer directly.	Consumers keep their own copy, so a slow or failed consumer never blocks the others or the submit request, and each retries on its own. Direct queries would add runtime coupling to C3 on every message and approval.